

PROYECTO PRX

Backlog de mantenimiento y defensa del Sprint 1

Notas, Bitácora y soporte de autenticación

Documento de planificación y defensa del mantenimiento evolutivo asignado al proyecto PRX. El trabajo funcional solicitado se limita a ampliar Notas con enlaces URL, tareas, imágenes y edición de imágenes, y ampliar Bitácora con enlaces URL y tareas.

Dato	Detalle
Entrega	Presentación y defensa del Sprint 1
Vencimiento	23 de septiembre de 2026 a las 23:59
Alcance funcional	Notas y Bitácora
Organización	Tres sprints incrementales
Tecnologías	Angular 21, PrimeNG, Fabric.js, NestJS, CQRS, Prisma y MySQL

Resultado esperado

Al finalizar los tres sprints, las notas podrán relacionar recursos mediante URL, manejar listas de tareas, adjuntar imágenes y modificarlas con herramientas gráficas. Las bitácoras podrán guardar enlaces y tareas, mostrar el detalle separado de la edición y persistir el marcado de tareas.

1 Alcance correcto del mantenimiento

Este mantenimiento no tiene como objetivo construir un Home, buscador general u otros módulos del producto. El backlog se deriva únicamente de las mejoras solicitadas para Notas y Bitácora. La autenticación se documenta como dependencia técnica porque aporta el usuario y los permisos, pero no constituye una historia funcional adicional del mantenimiento.

Módulo	Función solicitada	Resultado funcional
Notas	Enlaces URL	Agregar, validar, mostrar, abrir y eliminar enlaces relacionados.
Notas	Tareas	Crear, eliminar, visualizar y marcar tareas como completadas.
Notas	Imágenes	Adjuntar, previsualizar, quitar y guardar imágenes JPEG, PNG y WebP.
Notas	Editor de imágenes	Seleccionar, dibujar, marcar, recortar, rotar, añadir texto y formas, deshacer y rehacer.
Bitácora	Enlaces URL	Agregar, validar, mostrar, abrir y eliminar enlaces.
Bitácora	Tareas	Crear, eliminar, visualizar y marcar tareas con guardado inmediato.

2 Product Backlog dividido en tres sprints

La división minimiza dependencias: primero se establece el modelo de listas JSON y la interacción en Notas; después se incorpora el procesamiento gráfico; finalmente se reutilizan las decisiones aprendidas en Bitácora.

ID	Historia de usuario	Prioridad	Puntos	Sprint
HU M01	Como usuario quiero agregar enlaces URL a una nota para relacionarla con recursos externos.	Alta	5	1
HU M02	Como usuario quiero administrar tareas en una nota para controlar actividades pendientes y completadas.	Alta	8	1
HU M03	Como usuario quiero adjuntar imágenes a una nota para complementar visualmente su contenido.	Alta	5	2
HU M04	Como usuario quiero editar una imagen antes de guardarla para resaltar o corregir información.	Alta	13	2
HU M05	Como usuario quiero agregar enlaces URL a una bitácora para conservar referencias de mi trabajo.	Media	5	3
HU M06	Como usuario quiero administrar tareas en una bitácora para registrar el avance diario.	Alta	8	3

Resumen de sprints

Sprint	Objetivo	Historias	Puntos	Entregable
Sprint 1	Agregar enlaces URL y tareas en Notas.	HU M01 y HU M02	13	Notas relacionadas y accionables.
Sprint 2	Agregar imágenes y herramientas de edición en Notas.	HU M03 y HU M04	18	Flujo visual con editor gráfico.

Sprint	Objetivo	Historias	Puntos	Entregable
Sprint 3	Agregar enlaces URL y tareas en Bitácora.	HU M05 y HU M06	13	Bitácora enriquecida y editable.

3 Sprint 1 Enlaces URL y tareas en Notas

Objetivo: permitir que una nota contenga recursos web y una lista de actividades que pueda actualizarse desde la vista.

Valor: la nota deja de ser texto aislado y se convierte en un elemento de seguimiento y consulta.

Vencimiento: 23 de septiembre de 2026 a las 23:59.

3.1 Historia HU M01 Enlaces URL

Como usuario quiero agregar enlaces URL a una nota para acceder desde ella a documentación, sitios o recursos relacionados.

Criterios de aceptación

- El usuario puede escribir una dirección con o sin protocolo; el sistema normaliza el valor.
- Solo se aceptan direcciones HTTP o HTTPS válidas.
- Se pueden agregar varios enlaces y eliminar cualquiera antes de guardar.
- Los enlaces guardados se muestran en el detalle de la nota.
- Cada enlace se abre en una pestaña nueva con protección noopener noreferer.
- Los enlaces permanecen después de recargar la página.

3.2 Historia HU M02 Tareas

Como usuario quiero agregar y completar tareas dentro de una nota para registrar las actividades asociadas a su contenido.

Criterios de aceptación

- El usuario puede agregar una tarea con título no vacío.
- Puede eliminar tareas durante la edición.
- El detalle muestra tareas pendientes y completadas.
- El checkbox permite marcar o desmarcar una tarea sin entrar al modo de edición.
- El cambio se guarda inmediatamente mediante el endpoint de actualización.
- Las tareas permanecen después de recargar la página.

3.3 Sprint Backlog

Tarea	Trabajo técnico	Ubicación	Estado
T1	Definir NoteTaskModel y NoteLinkModel.	domain/models/note.model.ts	Hecho
T2	Crear controles para agregar y quitar URL.	create-note y note-detail HTML/TS	Hecho
T3	Normalizar y validar HTTP/HTTPS.	create-note y note-detail TS	Hecho
T4	Crear controles de tareas y checkbox.	note-detail HTML/TS	Hecho

Tarea	Trabajo técnico	Ubicación	Estado
T5	Enviar links y tasks mediante FormData.	infrastructure/api/note.api.ts	Hecho
T6	Validar y transformar JSON en backend.	UpdateNoteHandler	Hecho
T7	Persistir tasks y links como JSON.	schema.prisma y PrismaNoteRepository	Hecho
T8	Aplicar permisos de propietario o miembro.	Handlers de Notes	Hecho
T9	Probar guardado, recarga y URLs inválidas.	Prueba funcional	Verificado
T10	Preparar defensa y evidencia técnica.	Documento y demostración	Listo

3.4 Definición de terminado

- La interfaz permite administrar enlaces y tareas sin errores de maquetación.
- Frontend y backend compilan.
- Los datos se guardan y se recuperan desde MySQL mediante Prisma.
- Las entradas inválidas reciben mensajes comprensibles.
- Los permisos impiden modificar notas sin acceso al repositorio.
- La demostración completa puede repetirse antes de la entrega.

4 Evidencia técnica del Sprint 1

4.1 Modelo de dominio en Angular

Archivo: `prx-frontend/src/app/features/notes/domain/models/note.model.ts`

```
export interface NoteTaskModel {
  id: string;
  title: string;
  completed: boolean;
}

export interface NoteLinkModel {
  id: string;
  url: string;
}
```

Estas interfaces tipan los elementos antes de enviarlos al backend. El id permite localizar un elemento, title describe la tarea, completed conserva su estado y url contiene la dirección normalizada.

4.2 Validación de enlaces

Archivos: `create-note-page.component.ts` y `note-detail-page.component.ts`

```
private isValidUrl(value: string): boolean {
  try {
    const url = new URL(value);
    return url.protocol === 'http:' || url.protocol === 'https:';
  } catch {
    return false;
  }
}
```

La validación utiliza URL del navegador en lugar de expresiones regulares improvisadas. Además de comprobar la estructura, limita el protocolo para evitar valores no esperados.

4.3 Persistencia inmediata del checkbox

Archivo: `note-detail-page.component.ts`

```
protected toggleTask(task: NoteTaskModel): void {
  this.tasks.update((tasks) =>
    tasks.map((item) => item.id === task.id
      ? { ...item, completed: !item.completed }
      : item),
  );
  if (!this.editing()) this.persist(false);
}
```

La lista se actualiza de manera inmutable. Si el usuario está consultando la nota, `persist(false)` guarda el cambio sin abrir el editor, ofreciendo una interacción rápida.

4.4 Transporte y almacenamiento

Archivo: `prx-frontend/src/app/features/notes/infrastructure/api/note.api.ts`

```
formData.append('tasks', JSON.stringify(data.tasks));
formData.append('links', JSON.stringify(data.links));
```

Archivo: `prx-backend/prisma/schema.prisma`

```
model Note {
  title   String
  content String
  tasks   Json?
  links   Json?
}
```

Las listas se serializan porque la misma solicitud también transporta archivos. El backend vuelve a analizar y validar el JSON antes de guardarlo. Prisma utiliza campos `Json` opcionales porque las listas pertenecen a una nota y se recuperan junto con ella.

4.5 Archivos principales modificados

Capa	Archivo	Responsabilidad
Presentación	<code>notes-page.component.*</code>	Tarjetas, vista previa y navegación.
Presentación	<code>create-note-page.component.*</code>	Creación y enlaces URL.
Presentación	<code>note-detail-page.component.*</code>	Vista, edición, enlaces y tareas.
Aplicación	<code>note.facade.ts</code>	Coordinar API, loading y Store.
Dominio	<code>note.model.ts / requests</code>	Contratos tipados.
Infraestructura	<code>note.api.ts / note.store.ts</code>	HTTP, FormData y estado.
Backend	<code>notes.controller.ts</code>	Endpoints GET, POST, PUT y DELETE.
Backend	<code>UpdateNoteHandler</code>	Permisos, validación y caso de uso.
Persistencia	<code>schema.prisma / PrismaNoteRepository</code>	Campos JSON y acceso MySQL.

5 Sprint 2 Imágenes y editor en Notas

Objetivo: permitir adjuntar imágenes y modificarlas antes de guardarlas en la nota.

5.1 Historias y aceptación

Historia	Criterios de aceptación principales
HU M03 Imágenes	Aceptar JPEG, PNG y WebP; mostrar vista previa; permitir quitar; respetar el máximo conjunto de cinco archivos; guardar y volver a visualizar.
HU M04 Editor	Abrir cada imagen en un editor; aplicar herramientas; guardar como PNG; reemplazar la vista previa; conservar deshacer y rehacer.

5.2 Herramientas implementadas

Herramienta	Qué hace
Seleccionar	Mover, redimensionar o seleccionar objetos agregados.
Pincel	Dibujar trazos libres con color y grosor configurables.
Marcar	Aplicar resaltado semitransparente.
Recortar	Definir un área y reemplazar el lienzo con el recorte.
Texto	Agregar texto editable sobre la imagen.
Rotar 90 grados	Girar todo el resultado un cuarto de vuelta.
Formas	Agregar rectángulos, círculos y flechas.
Eliminar selección	Quitar el objeto gráfico activo.
Deshacer y rehacer	Recorrer el historial de cambios del lienzo.
Guardar cambios	Convertir el lienzo final en un archivo PNG.

5.3 Tareas del sprint

- Separar adjuntos generales de imágenes editables.
- Validar formatos JPEG, PNG y WebP y el máximo total de archivos.
- Crear previsualización con `URL.createObjectURL` y liberar recursos al eliminar.
- Integrar `Fabric.js` y un canvas adaptable.
- Implementar modos seleccionar, pincel, marcador y recorte.
- Agregar texto, rectángulo, círculo y flecha.
- Implementar rotación, historial, deshacer y rehacer.
- Exportar el resultado a PNG y sustituir la imagen editada.
- Permitir editar también una imagen ya guardada en una nota.

6 Sprint 3 Enlaces URL y tareas en Bitácora

Objetivo: enriquecer cada registro diario con referencias web y una lista de tareas persistente.

6.1 Historias y aceptación

Historia	Criterios de aceptación principales
HU M05 Enlaces	Agregar varias URL HTTP/HTTPS, eliminar, guardar, listar y abrir de forma segura.
HU M06 Tareas	Agregar, eliminar, marcar, desmarcar y conservar tareas después de recargar.

6.2 Tareas del sprint

- Añadir BinnacleTaskModel y BinnacleLinkModel.
- Incorporar controles de tareas y URL al formulario de creación.
- Agregar campos tasks y links al modelo Prisma Binnacle.
- Actualizar DTO, entidad, mapper y repositorio Prisma.
- Implementar GET por ID y PUT por ID.
- Separar la acción Ver de la acción Editar.
- Permitir marcar tareas desde la vista y guardar inmediatamente.
- Restringir el módulo al rol estandar y a registros propios.

6.3 Dependencias de seguridad

Bitácora utiliza @Roles(Role.estandar) y compara userId con el sub del JWT. Notas comprueba propiedad o membresía del repositorio. Estas reglas pertenecen al soporte de autenticación existente y permiten demostrar que las nuevas funciones no exponen información de otros usuarios.

7 SOLID aplicado al mantenimiento

Principio	Aplicación concreta
S Responsabilidad única	El componente maneja interacción; la Facade coordina; el Handler aplica reglas; el repositorio persiste.
O Abierto cerrado	Los contratos permiten ampliar implementaciones sin cambiar las páginas consumidoras.
L Sustitución de Liskov	Las implementaciones Prisma respetan los contratos NoteRepository y BinnacleRepository.
I Segregación de interfaces	Notas, Bitácora y Auth tienen contratos separados y enfocados.
D Inversión de dependencias	Los Handlers dependen de repositorios abstractos inyectados, no de Prisma directamente.

Ejemplo para defender

Al agregar tasks y links no se colocó toda la lógica en la pantalla. El modelo define la forma de los datos, NoteApi los transporta, UpdateNoteHandler valida permisos y contenido, y PrismaNoteRepository persiste. Cada cambio queda localizado en la capa que le corresponde.

8 Patrones utilizados

Patrón	Uso en el mantenimiento	Beneficio
--------	-------------------------	-----------

Patrón	Uso en el mantenimiento	Beneficio
CQRS	Commands para crear/actualizar/eliminar y Queries para consultar.	Casos de uso pequeños y localizables.
Repository	Contratos y repositorios Prisma para Note y Binnacle.	Desacopla negocio y base de datos.
Facade	NoteFacade y BinnacleFacade entre páginas, API y Store.	Simplifica la UI y centraliza loading.
Mapper	Conversión Prisma, dominio y respuestas.	Evita filtrar formatos entre capas.
DTO	Validación de entradas HTTP y JSON.	Rechaza datos incompletos o inválidos.
Guard	JWT, roles y guards de rutas Angular.	Centraliza autenticación y autorización.
Store reactivo	Signals para nota, bitácora, loading y error.	Estado predecible y reutilizable.
Soft delete	status igual a cero en lugar de borrado físico.	Conserva auditoría.

9 Calidad y puntualidad

9.1 Puntualidad

La entrega vence hoy, 23 de septiembre de 2026, a las 23:59. La defensa debe enviarse antes de esa hora. Para demostrar puntualidad conviene adjuntar este documento, registrar el envío en la plataforma y conservar una captura o comprobante con fecha y hora.

9.2 Evidencias de calidad

- Compilación Angular completada correctamente.
- Backend activo y rutas protegidas por JWT.
- Persistencia de tasks y links verificada en Bitácora.
- Validación HTTP/HTTPS en ambos módulos.
- Límites de archivos y formatos de imagen.
- Estados loading para evitar acciones duplicadas.
- Borrado lógico y campos de auditoría.
- Separación de vista y edición para reducir errores de uso.

9.3 Riesgos y siguientes mejoras

Prioridad	Mejora	Motivo
Alta	Agregar pruebas automatizadas.	Proteger validación, permisos y persistencia frente a regresiones.
Media	Agregar tareas durante la creación inicial de nota.	Actualmente se gestionan después desde detalle y edición.
Media	Optimizar imágenes grandes antes de guardarlas.	Reducir memoria, transferencia y almacenamiento.
Baja	Reducir el SCSS de create-note.	Eliminar la advertencia del presupuesto Angular.

10 Guion para presentar y defender el Sprint 1

Tiempo	Contenido	Evidencia
1 min	Alcance real	Notas: URL y tareas. Mostrar backlog y criterios.
2 min	Demostración URL	Agregar, guardar, recargar y abrir un enlace.
2 min	Demostración tareas	Crear, marcar desde la vista y recargar.
2 min	Arquitectura	Modelo, Facade, API, Handler y repositorio.
2 min	SOLID y patrones	SRP, DIP, CQRS, Repository, Facade y DTO.
1 min	Calidad	Validaciones, permisos, build y persistencia.
1 min	Cierre	Cumplimiento del Sprint 1 y próximos sprints.

Demostración paso a paso

1. Iniciar sesión y abrir una nota existente.
2. Entrar a Editar y agregar una URL relacionada.
3. Agregar una tarea y guardar la nota.
4. Volver a la vista y abrir el enlace en una pestaña nueva.
5. Marcar la tarea sin entrar nuevamente a Editar.
6. Recargar la página y comprobar que enlace y estado permanecen.
7. Mostrar `note.model.ts`, `note.api.ts`, `UpdateNoteHandler` y `schema.prisma`.
8. Cerrar explicando SOLID y los patrones usados.

Mensaje de apertura

El mantenimiento asignado se dividió en tres incrementos. En el Sprint 1 trabajamos enlaces URL y tareas en Notas. El resultado conecta la interfaz Angular con casos de uso CQRS y persistencia Prisma, manteniendo validación y permisos. Primero mostraremos el flujo funcional y luego explicaremos las decisiones de diseño.

Mensaje de cierre

El Sprint 1 cumple su objetivo: las notas guardan enlaces válidos y tareas que pueden completarse desde la vista. La solución aplica separación de responsabilidades y patrones que permiten continuar con imágenes en el Sprint 2 y reutilizar el enfoque en Bitácora durante el Sprint 3.

11 Preguntas probables y respuestas

¿Qué se pidió exactamente?

Agregar URL, tareas, imágenes y edición de imágenes en Notas; agregar URL y tareas en Bitácora. El Sprint 1 se concentra en URL y tareas de Notas.

¿Por qué se dividió así?

Las listas de enlaces y tareas crean primero el modelo y la persistencia; el editor gráfico tiene mayor complejidad y se separa; Bitácora reutiliza patrones ya probados.

¿Dónde está SOLID?

La UI, Facade, Handler y Repository tienen responsabilidades distintas. Los Handlers dependen de contratos de repositorio, lo que aplica inversión de dependencias.

¿Qué patrones usaron?

CQRS, Repository, Facade, Mapper, DTO, Guard, Store reactivo y soft delete.

¿Cómo validan una URL?

Se normaliza el protocolo y se utiliza new URL; solo se aceptan protocolos HTTP o HTTPS.

¿Cómo se guardan tareas y enlaces?

Se envían como JSON dentro de FormData, el backend los analiza y valida, y Prisma los guarda en campos Json.

¿Cómo se evita editar datos ajenos?

El JWT entrega el userId; los Handlers comprueban propiedad o membresía. Bitácora también exige el rol estandar.

¿Qué herramienta se usa para imágenes?

Fabric.js sobre canvas, con selección, pincel, marcador, recorte, rotación, texto, formas e historial.

¿Qué falta mejorar?

Principalmente pruebas automatizadas, optimización de imágenes grandes y permitir tareas en la creación inicial de una nota.

12 Conclusión

El backlog propuesto cubre exactamente el mantenimiento asignado y lo distribuye en tres incrementos defendibles. El Sprint 1 entrega valor visible mediante enlaces y tareas en Notas; el Sprint 2 incorpora imágenes y edición; el Sprint 3 lleva enlaces y tareas a Bitácora. La arquitectura existente permite desarrollar cada incremento sin concentrar responsabilidades ni debilitar los permisos.

Sprint 1: URL y tareas de Notas listos para presentación y defensa.

Sprint 2: imágenes y editor gráfico con Fabric.js.

Sprint 3: URL y tareas de Bitácora con control por rol y propiedad.

Anexo Archivos de referencia

- prx-frontend/src/app/features/notes
- prx-frontend/src/app/features/binnacles
- prx-frontend/src/app/core/guards e interceptors
- prx-backend/src/modules/notes
- prx-backend/src/modules/binnacles
- prx-backend/src/modules/auth
- prx-backend/prisma/schema.prisma